

Spectral bipartition

Spectral bipartition reads connectivity from the graph Laplacian

The idea

- Build L equals D minus A , take the second eigenvector, sort nodes
- Reject lopsided splits
- The continuous membership becomes a hard bipartition only after the sweep chooses the best

Pseudocode

- Spectral bipartition reads the Fiedler vector of the Laplacian
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- On the starter graph the winning split is D E versus A B C with cutsize three

Algorithm sketch — try these

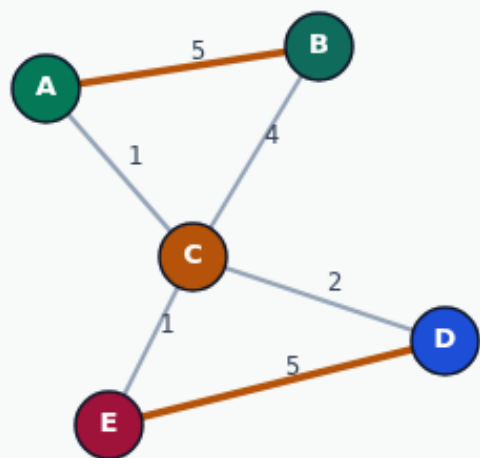
```
INPUT: weighted undirected G
OUTPUT: side[] bipartition
L ← Laplacian; take Fiedler eigenvector
order ← nodes sorted by Fiedler value
sweep balanced prefixes; pick min cutsize
GOLDEN: DE|ABC (or ABC|DE) cut=3
```

Build the Laplacian

SPECTRAL BIPARTITION

Build the Laplacian

Step 1 / 5 · laplacian · module02-05-spectral-partition



Explanation

Spectral methods read connectivity from the graph Laplacian $L = D - A$. The Fiedler vector (second eigenvector) encodes a soft cut.

- Tiny $n \rightarrow$ pure shifted inverse iteration
- No external linear-algebra library
- Same TINY_GRAPH as other labs

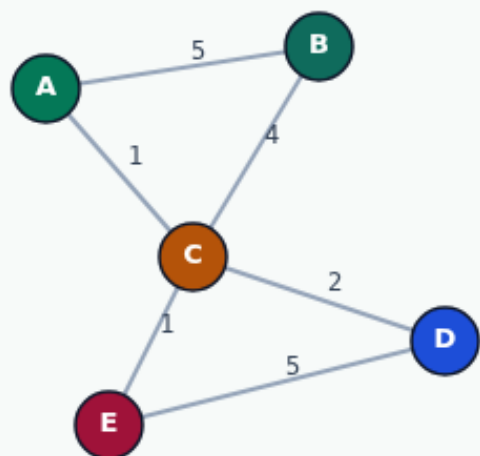
nodes : 5
edges : 6

Sort by Fiedler value

SPECTRAL BIPARTITION

Sort by Fiedler value

Step 2 / 5 · fiedler-order · module02-05-spectral-partition



Explanation

After iteration, nodes order low $\rightarrow$ high as E, D, C, B, A. E is most negative; A is most positive — natural bipartition candidates.

- Order endpoints: E lowest, A highest
- Values are continuous soft memberships
- Balance filter rejects lopsided prefixes

order: E < D < C < B < A

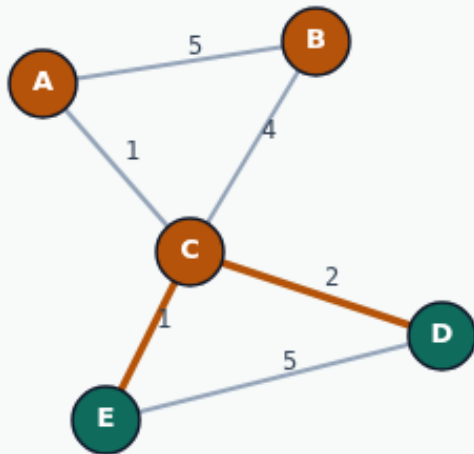
Sort by Fiedler value

Sweep prefixes for best cut

SPECTRAL BIPARTITION

Sweep prefixes for best cut

Step 3 / 5 · sweep-cut · module02-05-spectral-partition



Explanation

Try every balanced prefix of the order as side 0. The winning split is {D,E} vs {A,B,C} with cutsize 3.

- Balance window $\approx 20\text{--}80\%$ size
- Pick minimum cut among legal splits
- Golden cut = 3

best parts: DE | ABC
cutsize: 3

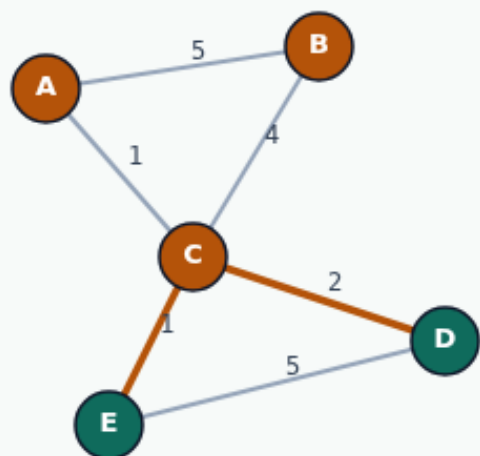
Sweep prefixes for best cut

Result ABC|DE

SPECTRAL BIPARTITION

Result ABC|DE

Step 4 / 5 · final · module02-05-spectral-partition



Explanation

Spectral recovers the same communities as unconstrained greedy and grow-from-D. Global eigenvectors find the cut without greedy merges.

- parts: ABC|DE
- Heavy edges internal
- Bridge C-D/C-E cut

cutsizes: 3
parts: ABC|DE

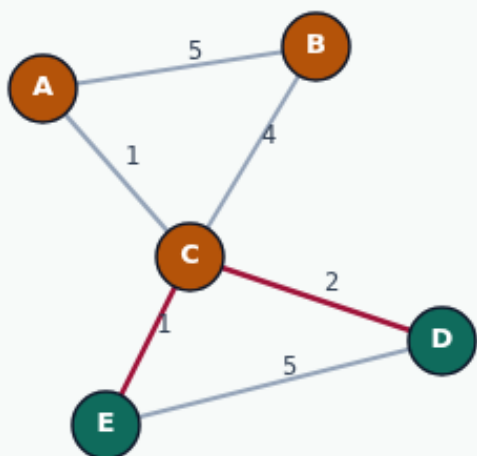
Result ABC|DE

When spectral helps

SPECTRAL BIPARTITION

When spectral helps

Step 5 / 5 · takeaway · module02-05-spectral-partition



Explanation

Spectral is a strong initializer for bipartition. On larger chips it pairs with multilevel; here the tiny instance grades against golden cut 3.

- Good seed for KL/FM
- Balance via sweep, not afterthought
- Used inside recursive bisection too

Golden: cut=3, E lowest / A highest

When spectral helps

Browser lab track

- In the browser lab track, open the **spectral-partition** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module